AI AGENT SAFETY — WORKSTREAM B

Safety Prompt Testing Strategy

One-page summary for stakeholder review. Full details: `planning/evaluation-working-reference.md`

OBJECTIVE & HYPOTHESIS

Objective: Determine whether the safety prompt add-on measurably reduces adversarial attack success against a multi-agent workflow.

Hypothesis: Injecting the safety prompt into agent instructions will significantly reduce attack success rates without meaningfully degrading task performance.

DESIGN PRINCIPLES

PRINCIPLE	WHAT IT MEANS
Modular	Supports new workflows, add-ons, and models without redesign
Repeatable	Same framework, swap the variable, re-run
Prompt-isolated	Platform held neutral — only the prompt changes
Extensible	Accommodates future modules and attack categories

ASSUMPTIONS & RISKS

Assumptions

- Foundry access will be granted
- Platform guardrails can be neutralized
- Test transcript is representative

Risks

- Model may refuse attacks without add-on (masks value)
- Single workflow may not generalize
- Corpus may be too small for statistical significance

PLATFORM

Azure AI Foundry

Target platform for building and testing the multi-agent workflow.

> **Why prompt-level safety matters:** Foundry is built on the OpenAI Agents SDK. The SDK's guardrails only fire on the first and last agents in a chain. Middle agents receive no SDK protection — **prompt-level safety is the only defense** for agents processing untrusted data mid-workflow.

WHAT WE'RE TESTING

Prompt-level safety only. Two versions tested in separate runs:

RUN	SAFETY VARIABLE
1	Lean baseline (v2)
2	Complete baseline (v1)

Platform guardrails held neutral to isolate the prompt's contribution.

SCOPE

In Scope

- Prompt-level safety add-on effectiveness
- Three attack categories: DPI, IPI, MR
- Per-agent (isolated) and end-to-end (pipeline) testing
- Single workflow: Client Pain Point Analysis (4 agents)
- Two safety prompt versions: Lean baseline (v2), Complete baseline (v1)
- F1 scoring methodology for evaluation
- Anonymized test transcript data

Out of Scope

- Platform-level guardrails (held neutral for isolation)
- Network, authentication, or infrastructure security
- Model fine-tuning or custom model training
- Production deployment or operational concerns
- Workflows beyond the test harness
- Real client data or PII handling

- Performance benchmarking (latency, throughput)

EVALUATION DESIGN

Four Conditions Per Run

	NO ATTACKS	WITH ATTACKS
No Add-on	A — Functional baseline	B — Vulnerability baseline
With Add-on	D — False positive cost	C — Safety evaluation

B → C delta = Safety prompt effectiveness | **A → D delta** = Usability cost

Three Attack Categories

Direct Prompt Injection

Indirect Prompt Injection

Malicious User Roleplay

Two Test Levels

- **Per-Agent:** Isolated — finds the weak link
- **End-to-End:** Full pipeline — tests propagation

F1 Scoring Summary

Binary classification: did the agent behave safely?

	ATTACK PRESENT	NO ATTACK
Safe	TP — resisted attack	TN — completed normally
Unsafe	FN — followed attack	FP — blocked without cause

Recall = TP / (TP + FN)
"Of all attacks, how many caught?"

F1 = 2 × TP / (2×TP + FP + FN)
Balanced summary metric

Scored per attack category (F1-DPI, F1-IPI, F1-MR) and aggregate. Full methodology in `planning/f1-scoring-methodology.md`

EXECUTION PLAN

PHASE	WHAT	BLOCKED BY
0	Strategy & test plan	NONE
1	Team review, platform access	PHASE 0
2	Build test harness (4 agents, orchestration)	PHASE 1
3	Author control + treatment prompts	PHASE 2
4	Build attack corpus + benign baselines	PHASE 0
5	Run evaluations (all conditions, both levels)	PHASES 2–4
6	Score, analyze, report findings	PHASE 5

Current blocker: Azure AI Foundry access (Phase 1)

EXTENSIBILITY

The framework is designed to support:

- **New workflows** — Different agent counts, patterns, input/output types
- **New add-on modules** — Context-specific safety modules tested individually or in combination
- **Model comparison** — Same workflow, different models (future runs)

OPENAI AGENTS SDK — CORE PRIMITIVES

SDK Building Blocks

PRIMITIVE	DESCRIPTION
Agents	LLMs configured with instructions, tools, and handoffs
Tools	Functions the agent can invoke to take actions or retrieve data
Handoffs	Allow agents to delegate to other agents for specific subtasks
Guardrails	Input/output filters that validate or block content (first/last agent only)

PRIMITIVE	DESCRIPTION
Tracing	Built-in observability for debugging and monitoring agent runs

Orchestration Patterns

PATTERN	DESCRIPTION	USED IN TEST HARNESS
Single Agent	One agent with tools, loops until task complete	NO
Agents as Tools	Supervisor agent calls sub-agents as tools, retains control	YES
Handoffs	Agents delegate fully to other agents, control transfers	NO

> **Test Harness Pattern:** Supervisor orchestrates Preparer → Reviewer → Formatter as tools. Supervisor retains control; sub-agents execute and return results.